OAuth 2.0: authenticate users with Google

OAuth2 (https://oauth.net/2/) is the industry-standard protocol for authorization. It provides a mechanism for users to grant web and desktop applications access to private information without sharing their username, password, and other private credentials.

This tutorial builds an extension that accesses a user's Google contacts using the Google People API (https://developers.google.com/people/) and the Chrome Identity API (/docs/extensions/reference/api/identity). Because extensions don't load over HTTPS, can't perform redirects or set cookies, they rely on the Chrome Identity API to use OAuth2.

Get started

Begin by creating a directory and the following starter files.

manifest.json

Add the manifest by creating a file called **`manifest.json`** and include the following code.

```
{
  "name": "OAuth Tutorial FriendBlock",
  "version": "1.0",
  "description": "Uses OAuth to connect to Google's People API and display conta
  "manifest_version": 3,
  "action": {
    "default_title": "FriendBlock, friends face's in a block."
  },
  "background": {
    "service_worker": "service-worker.js"
  }
```

```
}
```

service-worker.js

Add the extension service worker by creating a file called **`service-worker.js`** and include the following code.

```
chrome.action.onClicked.addListener(function() {
  chrome.tabs.create({url: 'index.html'});
});
```

index.html

Add an HTML file called **`index.html`** and include the following code.

```
<html>
  <head>
    <title>FriendBlock</title>
    <style>
      button {
        padding: 10px;
        background-color: #3C79F8;
        display: inline-block;
      }
    </style>
  </head>
  <body>
    <button>FriendBlock Contacts</button>
    <div id="friendDiv"></div>
  </body>
</html>
```

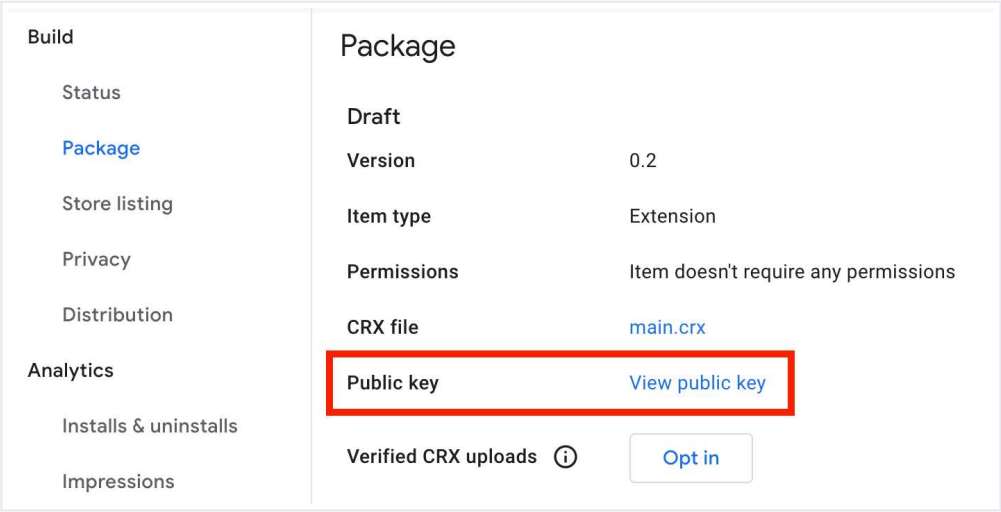
Keep a consistent extension ID

Preserving a single ID is essential during development. To keep a consistent ID, follow these steps:

Upload extension to the developer dashboard

Package the extension directory into a `.zip` file and upload it to the Chrome Developer Dashboard (https://chrome.google.com/webstore/developer/dashboard) without publishing it:

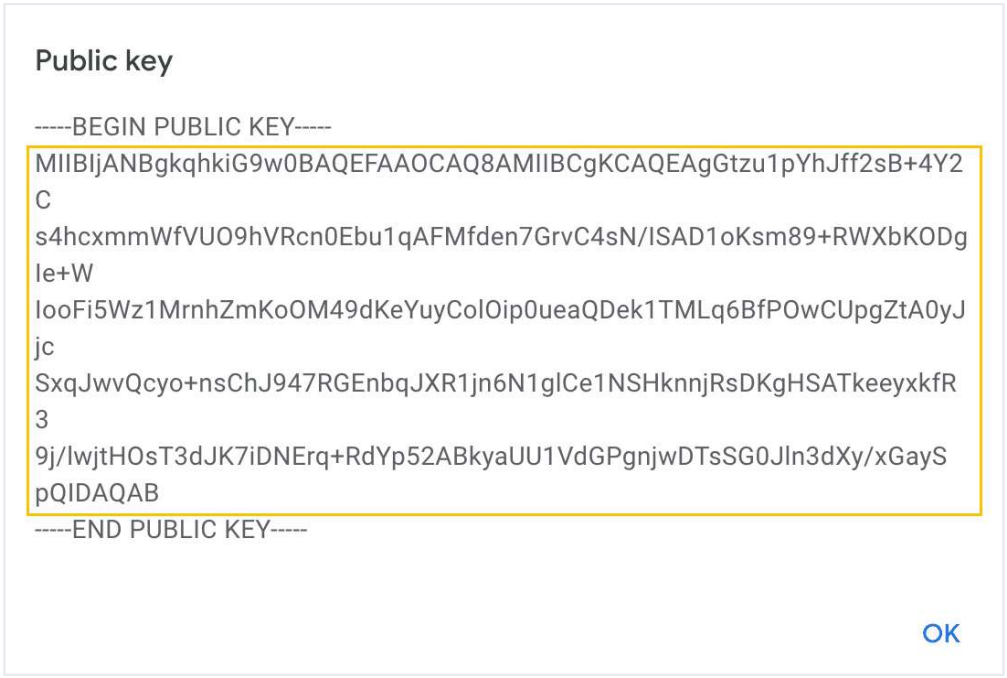
1. On the Developer Dashboard, click **Add new item**.
2. Click **Browse files**, select the extension's zip file, and upload it.
3. Go to the **Package** tab and click **View public key**.



View public key button in Package tab

When the dialog is open, follow these steps:

1. Copy the code in between `-----BEGIN PUBLIC KEY-----` and `-----END PUBLIC KEY-----`.
2. Remove the newlines in order to make it a single line of text.



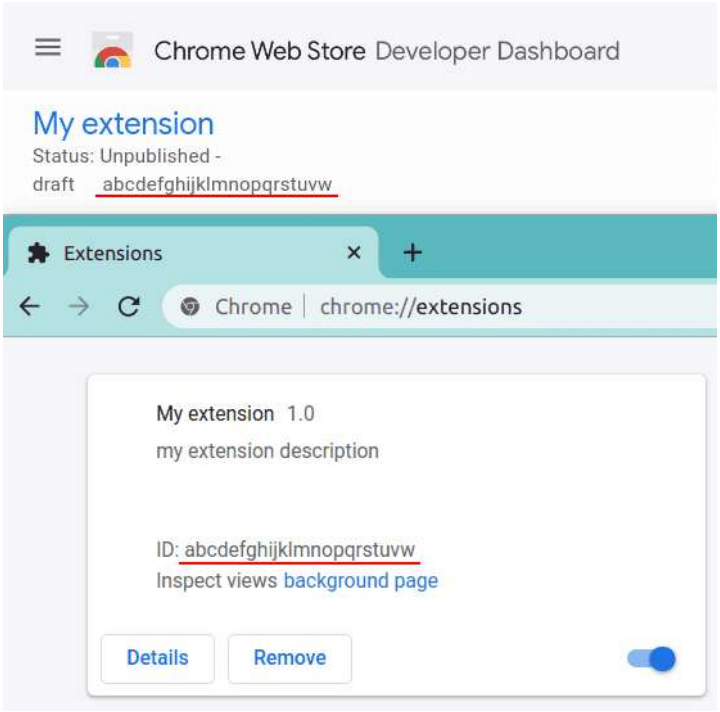
Public key dialog window

Add the code to the **`manifest.json`** under the **`"key"`** (/docs/extensions/reference/manifest/key) field. This way the extension will use the same ID.

```
{ // manifest.json
  "manifest_version": 3,
...
  "key": "ThisKeyIsGoingToBeVeryLong/go8GGC2u3UD9WI3MkmBgyiDPP2OreImEQhPvwpliioU
}
```

Compare IDs

Open the Extensions Management page at **`chrome://extensions`**, ensure **Developer mode** is enabled, and upload the unpackaged extension directory. Compare the extension ID on the extensions management page to the Item ID in the Developer Dashboard. They should match.



Create an OAuth client ID

Any application that uses OAuth 2.0 to access Google APIs must have authorization credentials that identify the application to Google's OAuth 2.0 server. The following steps explain how to create credentials for your project. Your applications can then use the credentials to access APIs that you have enabled for that project.

Start by navigating to the Google API console (https://console.developers.google.com/apis) to create a new project if you don't already have one. Follow these instructions to create an OAuth Client and obtain a Client ID.

1. Go to the Clients page (https://console.developers.google.com/auth/clients).
    a. Click **Create Client**.
    b. Select the **Chrome Extension** application type.
    c. Enter a name for the OAuth client. This name is displayed on your project's Clients page (https://console.developers.google.com/auth/clients) to identify the client.
    d. Enter the extension ID in the Item ID field.
    e. Click **Create**.

Register OAuth in the manifest

Include the "**oauth2**" field in the extension manifest. Place the generated OAuth client ID under "**client_id**". In order to get access to the user's account information, we need to request the relevant "**scope**", "**https://www.googleapis.com/auth/userinfo.email**".

```
{
  "name": "OAuth Tutorial FriendBlock",
  ...
  "oauth2": {
    "client_id": "yourExtensionOAuthClientIDWillGoHere.apps.googleusercontent.co
    "scopes":["https://www.googleapis.com/auth/userinfo.email"]
  },
  ...
}
```

Initiate first OAuth flow

Register the **identity** (/docs/extensions/reference/api/identity) permission in the manifest.

```
{
  "name": "OAuth Tutorial FriendBlock",
  ...
  "permissions": [
    "identity"
  ],
  ...
}
```

Create a file to manage the OAuth flow called **oauth.js** and include the following code.

```
window.onload = function() {
  document.querySelector('button').addEventListener('click', function() {
    chrome.identity.getAuthToken({interactive: true}, function(token) {
      console.log(token);
    });
  });
};
```

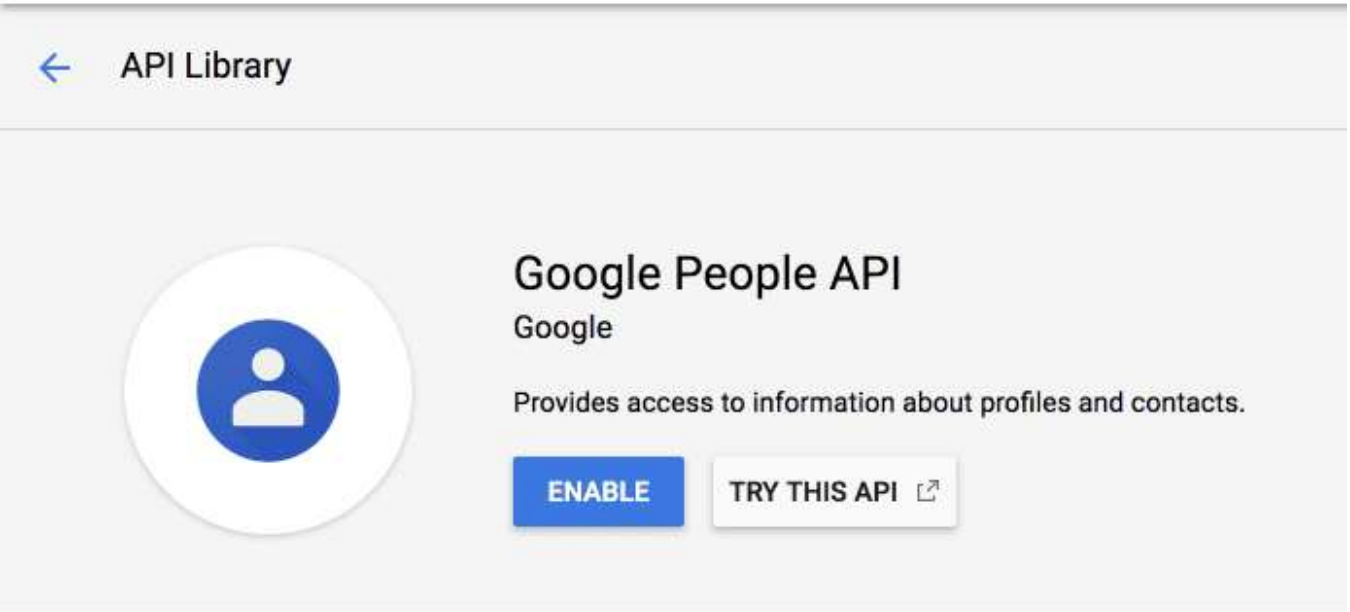
Place a script tag for **`oauth.js`** in the head of **`index.html`**.

```
...
  <head>
    <title>FriendBlock</title>
    ...
    <script type="text/javascript" src="oauth.js"></script>
  </head>
...
```

Reload the extension and click the browser icon to open **`index.html`**. Open the console and click the "FriendBlock Contacts" button. An OAuth token will appear in the console.

Enable the Google People API

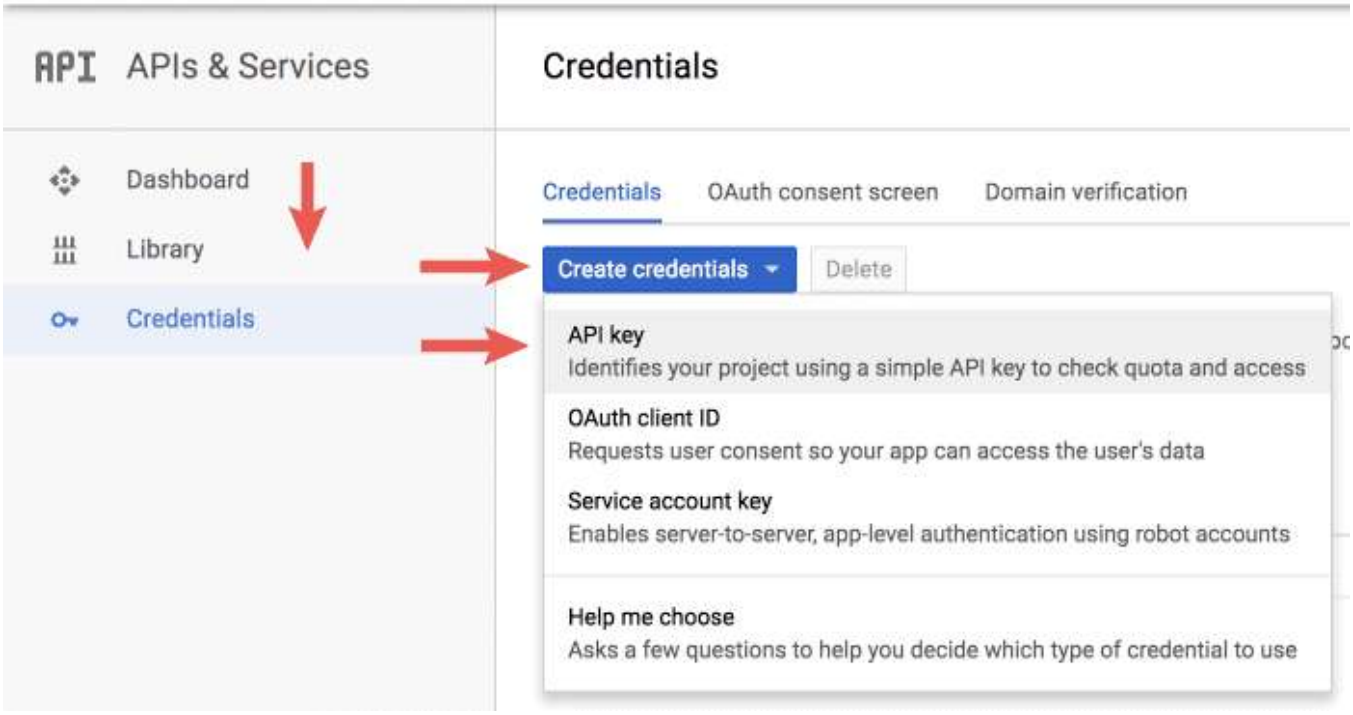
Return to the Google API console and select **Library** from the sidebar. Search for "Google People API", click the correct result and enable it.



Add the Google People API (https://developers.google.com/people/) client library to "**scopes**" in the extension manifest.

```
{
  "name": "OAuth Tutorial FriendBlock",
  ...
  "oauth2": {
    "client_id": "yourExtensionOAuthClientIDWillGoHere.apps.googleusercontent.co
    "scopes": [
      "https://www.googleapis.com/auth/contacts.readonly"
    ]
  },
  ...
}
```

Return to the Google API console and navigate back to credentials. Click "Create credentials" and select "API key" from the dropdown.



Keep the generated API key for later use.

Create the first API request

Now that the extension has proper permissions, credentials, and can authorize a Google user, it can request data through the People API. Update the code in **`oauth.js`** to match below.

```
window.onload = function() {
  document.querySelector('button').addEventListener('click', function() {
    chrome.identity.getAuthToken({interactive: true}, function(token) {
      let init = {
        method: 'GET',
        async: true,
        headers: {
          Authorization: 'Bearer ' + token,
          'Content-Type': 'application/json'
        },
        'contentType': 'json'
      };
      fetch(
          'https://people.googleapis.com/v1/contactGroups/all?maxMembers=20&key=
          init)
```

```
          .then((response) => response.json())
          .then(function(data) {
            console.log(data)
          });
    });
  });
};
```

Replace ***API_KEY*** with the API key generated from the Google API console. The extension should log a JSON object that includes an array of **`people/account_id`**s under the **`memberResourceNames`** field.

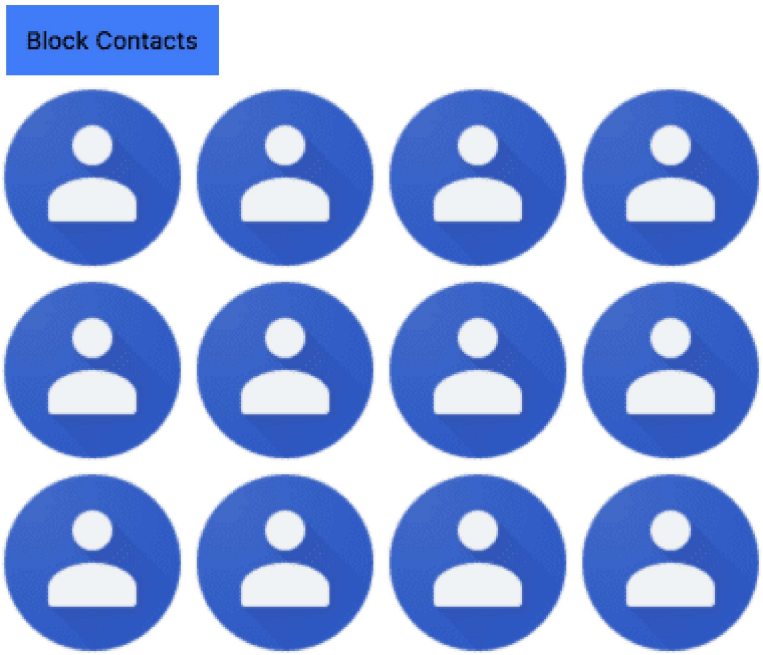
Block faces

Now that the extension is returning a list of the user's contacts, it can make additional requests to retrieve those contact's profiles and information (https://developers.google.com/people/api/rest/v1/contactGroups/get) . The extension will use the **`memberResourceNames`** to retrieve the photo information of user contacts. Update **`oauth.js`** to include the following code.

```
window.onload = function() {
  document.querySelector('button').addEventListener('click', function() {
    chrome.identity.getAuthToken({interactive: true}, function(token) {
      let init = {
        method: 'GET',
        async: true,
        headers: {
          Authorization: 'Bearer ' + token,
          'Content-Type': 'application/json'
        },
        'contentType': 'json'
      };
      fetch(
          'https://people.googleapis.com/v1/contactGroups/all?maxMembers=20&key=
          init)
          .then((response) => response.json())
```

```
          .then(function(data) {
            let photoDiv = document.querySelector('#friendDiv');
            let returnedContacts = data.memberResourceNames;
            for (let i = 0; i < returnedContacts.length; i++) {
              fetch(
                  'https://people.googleapis.com/v1/' + returnedContacts[i] +
                      '?personFields=photos&key=API_KEY',
                  init)
                  .then((response) => response.json())
                  .then(function(data) {
                    let profileImg = document.createElement('img');
                    profileImg.src = data.photos[0].url;
                    photoDiv.appendChild(profileImg);
                  });
            };
          });
    });
  });
};
```

Reload and return to the extension. Click the FriendBlock button and ta-da! Enjoy contact's faces in a block.



Last updated 2012-09-18 UTC.